

LAPORAN PRAKTIKUM 8

Nama : Akmallutfi Afza Gusty

NIM : 707012500064

Kelas : D4 SIKC 49-04

Mata kuliah : Pemrograman Web

Tujuan Praktikum

Setelah menyelesaikan praktikum ini, mahasiswa diharapkan mampu:

1. Memahami konsep dan fungsi framework dalam pengembangan aplikasi web.
2. Memahami arsitektur MVC (Model-View-Controller).
3. Menjelaskan arsitektur dan konsep dasar CodeIgniter 3.
4. Menginstal dan mengonfigurasi CodeIgniter 3.
5. Membuat routing dan controller pada CI3.
6. Mengimplementasikan view dan template system.
7. Menghubungkan model dengan database.
8. Mengintegrasikan komponen MVC menjadi aplikasi utuh.
9. Mengimplementasikan form handling dan validation.
10. Mengelola session, autentikasi, dan role management.
11. Menerapkan praktik keamanan dasar pada aplikasi CI3.

Alat dan bahan

Untuk melaksanakan praktikum ini, diperlukan:

1. Laptop/PC dengan XAMPP terinstal.
2. PHP versi minimal 7.x.
3. Database MySQL/MariaDB.
4. Code Editor (VS Code/Sublime/PHPStorm).
5. Browser (Chrome/Firefox).
6. CodeIgniter 3 (download dari <https://codeigniter.com>).

Langkah-Langkah Praktikum

View/proyek/index.php

```
<?php $page_title = 'Dashboard Proyek';  
$this->load->view('templates/header'); ?>  
  
<div class="container">  
    <div class="dashboard-header">  
        <h2>Laporan Kendala</h2>
```

```

        <div class="user-info">
            <span>Halo, <strong><?=
htmlspecialchars($username) ?></strong></span>
            <a href="<?= site_url('auth/logout') ?>"
class="logout-btn">Logout</a>
        </div>
    </div>

    <hr>

    <!-- Form Create / Update -->
    <form action="<?= site_url('proyek/simpan') ?>"
method="post" enctype="multipart/form-data">
        <input type="hidden" name="id" value="<?=
$edit_data['id'] ?>">
        <input type="hidden" name="file_lama"
value="<?= $edit_data['nama_file'] ?>">

        <input type="text"
            name="nama_proyek"
            placeholder="Nama Proyek"
            value="<?=
htmlspecialchars($edit_data['nama_proyek']) ?>"
            required>

        <textarea name="deskripsi"
            placeholder="Deskripsi Proyek"><?=
htmlspecialchars($edit_data['deskripsi']) ?></textarea>

        <input type="file" name="berkas">

        <button type="submit" name="simpan">
            <?= $edit_data['id'] ? 'UPDATE DATA' :
'SIMPAN DATA & UPLOAD' ?>
        </button>
    </form>

    <hr>

```

```

<!-- Tabel Daftar Proyek -->
<table>
    <thead>
        <tr>
            <th>No</th>
            <th>Nama Kendala</th>
            <th>Deskripsi</th>
            <th>File</th>
            <th>Aksi</th>
        </tr>
    </thead>
    <tbody>
        <?php if (empty($proyek)): ?>
            <tr>
                <td colspan="5" style="text-align:center;">Belum ada data laporan.</td>
            </tr>
        <?php else: ?>
            <?php $no = 1;
            foreach ($proyek as $row): ?>
                <tr>
                    <td><?= $no++ ?></td>
                    <td><?=
htmlspecialchars($row['nama_proyek']) ?></td>
                    <td><?=
htmlspecialchars($row['deskripsi']) ?></td>
                    <td>
                        <?php if
($row['nama_file']): ?>
                            <a href="<?=
base_url('uploads/' . $row['nama_file']) ?>"
target="_blank">
                                <?=
htmlspecialchars($row['nama_file']) ?>
                            </a>
                        <?php else: ?>
                            <em>--</em>

```

```

                <?php endif; ?>
            </td>
            <td class="aksi">
                <a href="<?=
site_url('proyek?edit=' . $row['id']) ?>" class="btn-
edit">Edit</a>
                <a href="<%=
site_url('proyek/hapus/' . $row['id']) ?>"
                    class="btn-hapus"
                    onclick="return
confirm('Yakin ingin menghapus proyek ini?')">Hapus</a>
            </td>
        </tr>
    <?php endforeach; ?>
    <?php endif; ?>
</tbody>
</table>
</div>

</body>

</html>

```

Auth/login.php

```

<?php $page_title = 'Login'; $this->load-
>view('templates/header'); ?>

<div class="container">
    <h2>Masuk</h2>

    <?php if ($error): ?>
        <p class="msg-error"><%=
htmlspecialchars($error) ?></p>
    <?php endif; ?>

    <form method="post" action="<%=
site_url('auth/login') ?>">
        <input type="text"

```

```

        name="username"
        placeholder="Username"
        required
        autocomplete="username">

        <input type="password"
        name="password"
        placeholder="Password"
        required
        autocomplete="current-password">

        <button type="submit"
name="login">Login</button>

        <p>Belum punya akun? <a href="<?=
site_url('auth/register') ?>">Daftar di sini</a></p>
    </form>
</div>

</body>
</html>

```

Auth/register.php

```

<?php $page_title = 'Daftar Akun'; $this->load-
>view('templates/header'); ?>

<div class="container">
    <h2>Daftar Akun Baru</h2>

    <?php if ($error): ?>
        <p class="msg-error"><?=
htmlspecialchars($error) ?></p>
    <?php endif; ?>

    <?php if ($sukses): ?>
        <p class="msg-sukses"><?=
htmlspecialchars($sukses) ?></p>
    <?php endif; ?>

```

```

        <?php endif; ?>

        <form method="post" action="<?=
site_url('auth/register') ?>">
            <input type="text"
                name="username"
                placeholder="Username"
                required
                autocomplete="username">

            <input type="password"
                name="password"
                placeholder="Password"
                required
                autocomplete="new-password">

            <input type="password"
                name="confirm_password"
                placeholder="Ulangi Password"
                required
                autocomplete="new-password">

            <button type="submit" name="register">DAFTAR
SEKARANG</button>

            <p>Sudah punya akun? <a href="<?=
site_url('auth/login') ?>">Login di sini</a></p>
        </form>
    </div>

</body>
</html>

```

Model/Proyek_model.php

```

<?php
defined('BASEPATH') OR exit('No direct script access
allowed');

```

```

/**
 * Model Proyek_model
 *
 * Bertanggung jawab atas semua operasi CRUD pada tabel
 'proyek',
 * termasuk proses upload file berkas.
 *
 * Menggunakan CI3 Query Builder ($this->db) dan Upload
 Library
 * ($this->load->library('upload')) sebagai pengganti
 fungsi manual.
 */
class Proyek_model extends CI_Model {

    // Folder tujuan upload file (relatif terhadap root
    CI)
    private string $upload_path = './uploads/';

    /**
     * Mengambil semua data proyek dari database.
     *
     * @return array hasil query semua proyek
     */
    public function get_all(): array
    {
        return $this->db->get('proyek')->
>result_array();
    }

    /**
     * Mengambil satu data proyek berdasarkan ID (untuk
 mode Edit).
     *
     * @param int $id
     * @return array data proyek sebagai associative
 array
     */

```



```

public function get_by_id(int $id): array
{
    $this->db->where('id', $id);
    return $this->db->get('proyek')->row_array();
}

/**
 * Menambahkan proyek baru ke database (Create).
 * Menangani upload file jika ada berkas yang di-
upload.
 *
 * @param string $nama_proyek
 * @param string $deskripsi
 * @return bool
 */
public function create(string $nama_proyek, string
$deskripsi): bool
{
    $nama_file = $this->upload_file();

    return $this->db->insert('proyek', [
        'nama_proyek' => $nama_proyek,
        'deskripsi'   => $deskripsi,
        'nama_file'    => $nama_file,
    ]);
}

/**
 * Memperbarui data proyek (Update).
 * Jika ada file baru, upload dan gunakan nama file
baru;
 * jika tidak, pertahankan nama file lama.
 *
 * @param int $id
 * @param string $nama_proyek
 * @param string $deskripsi
 * @param string $file_lama Nama file lama
(fallback jika tidak ada upload baru)

```

```

        * @return bool
        */
        public function update(int $id, string
$nama_proyek, string $deskripsi, string $file_lama):
bool
        {
            // Coba upload file baru; jika tidak ada, tetap
            pakai file lama
            $nama_file = $_FILES['berkas']['name']
                ? $this->upload_file()
                : $file_lama;

            $this->db->where('id', $id);
            return $this->db->update('proyek', [
                'nama_proyek' => $nama_proyek,
                'deskripsi'    => $deskripsi,
                'nama_file'    => $nama_file,
            ]);
        }

    /**
     * Menghapus proyek berdasarkan ID.
     *
     * @param int $id
     * @return bool
     */
    public function delete(int $id): bool
    {
        $this->db->where('id', $id);
        return $this->db->delete('proyek');
    }

    /**
     * Method private untuk proses upload file
     menggunakan CI Upload Library.
     * Dipanggil oleh create() dan update().
     *

```

```

        * @return string Nama file yang berhasil di-
upload, atau string kosong
        */
        private function _upload_file(): string
        {
            // Jika tidak ada file yang di-upload,
            kembalikan string kosong
            if (empty($_FILES['berkas']['name'])) {
                return '';
            }

            $this->load->library('upload', [
                'upload_path' => $this->upload_path,
                'allowed_types' => '*',
                'overwrite' => TRUE,
            ]);

            if ($this->upload->do_upload('berkas')) {
                return $this->upload->data('file_name');
            }

            return '';
        }
    }
    ?>

```

Model/User_model.php

```

<?php
defined('BASEPATH') OR exit('No direct script access
allowed');

/**
 * Model User_model
 *
 * Bertanggung jawab atas semua query database yang
berhubungan
 * dengan tabel 'users': login dan registrasi akun.

```

```

*
* Dalam CI3, model mewarisi CI_Model dan menggunakan
$this->db
* sebagai pengganti objek koneksi manual (tidak perlu
Database.php lagi).
*/
class User_model extends CI_Model {

    /**
     * Melakukan proses login.
     * Mengambil data user berdasarkan username, lalu
memverifikasi
     * password menggunakan password_verify() terhadap
hash di database.
     *
     * @param string $username
     * @param string $password
     * @return bool true jika login berhasil
     */
    public function login(string $username, string
$password): bool
    {
        $this->db->where('username', $username);
        $query = $this->db->get('users');
        $user = $query->row_array();

        if ($user && password_verify($password,
$user['password'])) {
            // Simpan data sesi user
            $this->session->set_userdata([
                'login' => TRUE,
                'username' => $user['username'],
            ]);
            return TRUE;
        }

        return FALSE;
    }
}

```

```

/**
 * Melakukan proses registrasi akun baru.
 * Mengecek duplikasi username, kecocokan password,
 lalu insert ke DB.
 *
 * @param string $username
 * @param string $password
 * @param string $confirm_password
 * @return string 'sukses' atau pesan error
 */
public function register(string $username, string
$password, string $confirm_password): string
{
    // Cek apakah username sudah terdaftar
    $this->db->where('username', $username);
    if ($this->db->get('users')->num_rows() > 0) {
        return 'Username sudah terdaftar!';
    }

    // Cek kecocokan password
    if ($password !== $confirm_password) {
        return 'Konfirmasi password tidak cocok!';
    }

    // Hash password sebelum disimpan (keamanan
data)
    $hashed = password_hash($password,
PASSWORD_DEFAULT);

    // Insert user baru
    $insert = $this->db->insert('users', [
        'username' => $username,
        'password' => $hashed,
    ]);

    return $insert ? 'sukses' : 'Gagal mendaftarkan
user.';

```

```
}  
}  
?>
```

Controllers/ Auth.php

```
<?php  
defined('BASEPATH') OR exit('No direct script access  
allowed');  
  
/**  
 * Controller Auth  
 *  
 * Menangani proses autentikasi pengguna:  
 * - index()      : Redirect ke login  
 * - login()      : Tampilkan form login & proses POST  
 * - register()   : Tampilkan form register & proses POST  
 * - logout()     : Hapus sesi & redirect ke login  
 *  
 * URI: /auth/login | /auth/register | /auth/logout  
 */  
class Auth extends CI_Controller {  
  
    public function __construct()  
    {  
        parent::__construct();  
        $this->load->model('User_model');  
    }  
  
    /**  
     * Default method – redirect ke halaman login.  
     */  
    public function index()  
    {  
        redirect('auth/login');  
    }  
  
    /**
```

```

    * Halaman & proses Login.
    * GET → tampilkan view login
    * POST → validasi input, panggil
User_model::login()
    */
    public function login()
    {
        // Jika sudah login, langsung ke dashboard
        if ($this->session->userdata('login')) {
            redirect('proyek');
        }

        $data = ['error' => ''];

        if ($this->input->post('login')) {
            $username = $this->input->post('username',
TRUE);
            $password = $this->input->post('password');

            if ($this->User_model->login($username,
$password)) {
                redirect('proyek');
            } else {
                $data['error'] = 'Username atau
password salah!';
            }
        }

        $this->load->view('auth/login', $data);
    }

    /**
    * Halaman & proses Register.
    * GET → tampilkan view register
    * POST → validasi input, panggil
User_model::register()
    */
    public function register()

```

```

{
    // Jika sudah login, langsung ke dashboard
    if ($this->session->userdata('login')) {
        redirect('proyek');
    }

    $data = ['error' => '', 'sukses' => ''];

    if ($this->input->post('register')) {
        $username      = $this->input-
>post('username', TRUE);
        $password      = $this->input-
>post('password');
        $confirm_password = $this->input-
>post('confirm_password');

        $hasil = $this->User_model-
>register($username, $password, $confirm_password);

        if ($hasil === 'sukses') {
            $data['sukses'] = 'Registrasi berhasil!
Silakan login.';
        } else {
            $data['error'] = $hasil;
        }
    }

    $this->load->view('auth/register', $data);
}

/**
 * Logout – hapus semua data sesi & redirect ke
login.
 */
public function logout()
{
    $this->session->sess_destroy();
    redirect('auth/login');
}

```



```
}  
}  
?>
```

Controller/Welcome.php

```
<?php  
defined('BASEPATH') OR exit('No direct script access  
allowed');  
  
/**  
 * Controller Proyek  
 *  
 * Menangani semua fitur dashboard dan CRUD proyek:  
 * - index()      : Tampilkan daftar proyek + form  
tambah/edit  
 * - simpan()     : POST → Create atau Update proyek  
 * - hapus($id)  : GET  → Hapus proyek berdasarkan ID  
 *  
 * URI: /proyek | /proyek/simpan | /proyek/hapus/{id}  
 *  
 * Dilindungi auth-guard di __construct():  
 * user yang belum login akan di-redirect ke halaman  
login.  
 */  
class Proyek extends CI_Controller {  
  
    public function __construct()  
    {  
        parent::__construct();  
  
        // Auth Guard: pastikan user sudah login  
        if ( ! $this->session->userdata('login')) {  
            redirect('auth/login');  
        }  
  
        $this->load->model('Proyek_model');  
    }  
}
```

```

/**
 * Dashboard – menampilkan form tambah/edit dan
tabel daftar proyek.
 *
 * Jika ada ?edit=ID di URL, form akan terisi data
proyek untuk diedit.
 */
public function index()
{
    // Data default untuk form kosong (mode Create)
    $edit_data = [
        'id'            => '',
        'nama_proyek'   => '',
        'deskripsi'     => '',
        'nama_file'     => '',
    ];

    // Mode Edit: ambil data proyek berdasarkan ID
    $id = (int) $this->input->get('edit');
    if ($id > 0) {
        $edit_data = $this->Proyek_model-
>get_by_id($id);
    }

    $data = [
        'username'      => $this->session-
>userdata('username'),
        'proyek'        => $this->Proyek_model-
>get_all(),
        'edit_data'     => $edit_data,
    ];

    $this->load->view('proyek/index', $data);
}

/**
 * Proses simpan – Create atau Update proyek.

```

```

    * Hanya menerima POST request.
    */
    public function simpan()
    {
        if ( ! $this->input->post('simpan')) {
            redirect('proyek');
        }

        $id          = (int)    $this->input->
>post('id');
        $nama         = $this->input->post('nama_proyek',
TRUE);
        $deskripsi    = $this->input->
>post('deskripsi',    TRUE);
        $file_lama    = $this->input->
>post('file_lama',    TRUE);

        if ($id > 0) {
            // Mode Update
            $this->Proyek_model->update($id, $nama,
$deskripsi, $file_lama);
        } else {
            // Mode Create
            $this->Proyek_model->create($nama,
$deskripsi);
        }

        redirect('proyek');
    }

    /**
     * Hapus proyek berdasarkan ID.
     *
     * @param int $id
     */
    public function hapus(int $id = 0)
    {
        if ($id > 0) {

```

```
        $this->Proyek_model->delete($id);
    }
    redirect('proyek');
}
?>
```

Analisis dan Pembahasan

MVC adalah sebuah pola arsitektur perangkat lunak (*software architecture pattern*) yang membagi sebuah aplikasi menjadi tiga komponen utama yang saling terpisah namun saling berkomunikasi: ****Model****, ****View****, dan ****Controller****. Tujuan utama dari pola ini adalah memisahkan tanggung jawab (*separation of concerns*) agar kode menjadi lebih terstruktur, mudah dipelihara, dan mudah dikembangkan. Dalam konteks proyek CodeIgniter yang sedang kamu kerjakan, MVC adalah fondasi utama cara framework ini bekerja.

****Model**** adalah komponen yang bertanggung jawab atas semua urusan data dan logika bisnis aplikasi. Model berkomunikasi langsung dengan database — baik mengambil, menyimpan, memperbarui, maupun menghapus data. Model tidak peduli bagaimana data tersebut akan ditampilkan; tugasnya hanya menyediakan data yang benar. Contohnya adalah `User\_model.php` yang kamu miliki, yang bertugas mengambil data pengguna dari database untuk keperluan autentikasi.

****View**** adalah komponen yang bertugas menampilkan data kepada pengguna, alias lapisan antarmuka (*user interface*). View hanya mengetahui cara merender atau menampilkan data yang sudah diberikan kepadanya; ia tidak memproses logika apapun dan tidak mengakses database secara langsung. Dalam proyek kamu, file seperti `login.php` di folder `views/auth/` adalah contoh View — ia hanya menampilkan form login kepada pengguna.

****Controller**** adalah komponen penghubung antara Model dan View. Ketika pengguna melakukan sebuah aksi (misalnya mengklik tombol login), permintaan tersebut pertama kali masuk ke Controller. Controller kemudian memanggil Model yang sesuai untuk mendapatkan atau memproses data, lalu meneruskan hasilnya ke View yang tepat untuk ditampilkan. Controller adalah "otak" yang mengatur alur kerja aplikasi. Ia menerima

input, memproses permintaan, namun tidak mengandung logika data maupun logika tampilan secara langsung.

Secara sederhana, alur kerjanya adalah: **Pengguna** → **Controller** → **Model** → **Controller** → **View** → **Pengguna**. Pengguna berinteraksi dengan antarmuka (View), Controller menerima aksi tersebut, Model mengolah data, Controller menerima hasilnya, lalu View menampilkan respons kepada pengguna kembali. Dengan pola ini, jika kamu ingin mengubah tampilan aplikasi, kamu cukup mengubah View tanpa harus menyentuh logika data di Model, dan sebaliknya. Inilah yang membuat MVC sangat populer dalam pengembangan aplikasi web modern seperti CodeIgniter.

Kesimpulan

MVC (*Model-View-Controller*) adalah pola arsitektur yang membagi aplikasi web menjadi tiga lapisan dengan tanggung jawab yang jelas dan terpisah. **Model** mengelola data dan logika bisnis, **View** menangani tampilan antarmuka pengguna, dan **Controller** menjadi jembatan yang mengatur alur kerja di antara keduanya. Dengan pemisahan ini, setiap bagian aplikasi dapat dikembangkan, diuji, dan diperbaiki secara mandiri tanpa mengganggu komponen lainnya.

Penerapan pola MVC — terutama dalam framework seperti **CodeIgniter** — membawa banyak manfaat nyata: kode menjadi lebih rapi dan terstruktur, kolaborasi antar developer menjadi lebih mudah karena setiap orang bisa fokus pada satu lapisan saja, serta proses *debugging* dan pemeliharaan aplikasi menjadi jauh lebih efisien. Selain itu, MVC mendorong penulisan kode yang tidak redundan (*Don't Repeat Yourself*), sehingga fitur yang sama tidak perlu ditulis ulang di banyak tempat.

Singkatnya, MVC bukan sekadar aturan teknis, melainkan sebuah **filosofi desain** yang memastikan aplikasi yang kamu bangun saat ini dapat tumbuh dan berkembang dengan sehat di masa mendatang tanpa menjadi berantakan. Itulah mengapa hampir seluruh framework web modern, termasuk CodeIgniter, Laravel, hingga Django dan Ruby on Rails, mengadopsi pola ini sebagai standarnya.